

OCaml - Predavanje 8

Ponavljanje prethodnog gradiva — Closures (nastavak)

Closure u record-u

Zadatak:

Implementirati tip `banka` kao record koji simulira bankovni racun koristeći closure. Record treba imati tri polja:

- `napravi_transakciju: int -> banka` — closure koja prima iznos transakcije i vraća novi record sa azuriranim stanjem
- `stanje: int` — tekuće stanje racuna (suma svih dosad izvršenih transakcija)
- `broj_transakcija: int` — ukupan broj izvršenih transakcija

Inicijalno stanje racuna je 0 sa 0 transakcija. Svaka transakcija dodaje iznos na stanje. Closure interno pamti `lista_transakcija` iz svog okruženja — ta lista nije dostupna korisniku.

Closure se može smjestiti kao **polje record-a** — ovo simulira objektno-orijentiran stil u OCaml-u.

```
type banka = {  
  napravi_transakciju: int -> banka;  
  stanje: int;  
  broj_transakcija: int  
};;  
  
let rec moja_bank_a =  
  let rec napravi_banku lista_transakcija =  
    {  
      napravi_transakciju = (fun x -> napravi_banku (x :: lista_transakcija));  
      stanje = (List.fold_left (fun x y -> x + y) 0 lista_transakcija);  
      broj_transakcija = List.length lista_transakcija  
    }  
  in napravi_banku [];;
```

Primjer korištenja:

```
let moja_bank_a = moja_bank_a.napravi_transakciju 5;;  
moja_bank_a.stanje;;      (* = 5 *)  
moja_bank_a.broj_transakcija;;  (* = 1 *)  
  
let moja_bank_a = moja_bank_a.napravi_transakciju 3;;  
moja_bank_a.stanje;;      (* = 8 *)  
moja_bank_a.broj_transakcija;;  (* = 2 *)
```

Ideja: `napravi_transakciju` je closure koja pamti `lista_transakcija` iz okruženja. Svaki poziv kreira novi record sa prosirenom listom — banka je **nepromjenjiva** (immutable), ali polje-funkcija omogućava "tranziciju" u novo stanje.

Moduli i Module Types (signature)

1. Šta je modul?

Modul je **imenovan prostor (namespace)** koji grupira tipove, vrijednosti, izuzetke i funkcije. Omogućava organizaciju koda i **skrivanje implementacijskih detalja** od korisnika.

```
module Ime = struct
  (* tipovi, vrijednosti, funkcije ... *)
end;;
```

Pristup članovima modula vrši se operatorom `..`:

```
Modul.funkcija argument
```

2. Module type (signatura)

Module type definise **javni interfejs** modula — šta je vidljivo izvana. Sve što **nije** u signaturi automatski postaje **privatno** i nedostupno korisniku.

```
module type ImeTipa = sig
  type t
  exception MojIzuzetak of string
  val funkcija: int -> t
end;;
```

3. Modul sa eksplicitnom signaturom

```
module Naziv : ImeTipa = struct
  (* implementacija — mora zadovoljiti signaturu *)
end;;
```

Kompajler provjerava da modul implementira sve što signatura zahtijeva. Članovi koji nisu u signaturi su **automatski privatni**.

4. Skrivanje implementacije

Ključna prednost modula: može se mijenjati **implementacija** bez promjene **interfejsa**.

```
Korisnik --> [ Signatura: javno (sig) ] --> [ Implementacija: privatno (struct) ]
```

Primjer — tip `datum` je **apstraktan** u signaturi:

```
module type DatumTip = sig
  type datum (* korisnik ne zna šta je unutra *)
  val kreiraj_datum: int * int * int -> datum
end;;
```

Korisnik ne može direktno pisati `{dan=1; mjesec=1; godina=2020}` — mora koristiti `kreiraj_datum`, čime se provodi validacija.

Zadaci

Zadatak 1 — Modul Datum

Implementirati modul `Datum` sa odgovarajucim module type-om `DatumTip`. Onemoguciti korisniku kreiranje nevalidnih datuma (npr. 32.03.1990.). Implementirati sljedece funkcije:

- `da_li_je_prestupna: int -> bool` — vraca da li je proslijedjena godina prestupna
- `kreiraj_datum: int*int*int -> datum` — kreira datum ili baca izuzetak `InvalidanDatum` ako je datum nevalidan
- `datum_to_string: datum -> string` — formatira datum u string (npr. "10. Mart 2020.")
- `koji_dan: datum -> string` — vraca naziv dana u sedmici koristeći Zellerov algoritam (Ponedjeljak, Utorak, ...)

Pomocne funkcije `broj_dana_u_mjesecu` i `validan_datum` ne smiju biti vidljive korisniku.

Rjesenje: [Datum.ml](#)

```
module type DatumTip = sig
  type datum
  val da_li_je_prestupna: int -> bool
  exception InvalidanDatum of string
  val kreiraj_datum: int*int*int -> datum
  val datum_to_string: datum -> string
  val koji_dan: datum -> string
end;;

module Datum:DatumTip = struct
  type datum = {dan: int; mjesec: int; godina:int}
  exception InvalidanDatum of string

  let da_li_je_prestupna(godina) =
    if (godina mod 4 = 0 && godina mod 100 != 0) || (godina mod 400 = 0)
    then true
    else false

  let broj_dana_u_mjesecu((mjesec: int), (prestupna:bool)) =
    match mjesec with
    | 1 -> 31
    | 2 -> if prestupna then 29 else 28
    | 3 -> 31
    | 4 -> 30
    | 5 -> 31
    | 6 -> 30
    | 7 -> 31
    | 8 -> 31
    | 9 -> 30
    | 10 -> 31
```

```

| 11-> 30
| 12-> 31
| _ -> failwith("Nije validan mjesec")

let validan_datum(dan, mjesec, godina) =
  if mjesec < 1 || mjesec > 12 then false
  else if godina < 0 then false
  else if dan <= 0 then false
  else if dan > broj_dana_u_mjesecu(mjesec, da_li_je_prestupna(godina))
  then false
  else true

let kreiraj_datum(d, m, g) =
  if(validan_datum(d, m, g)) then
    {dan=d;mjesec=m;godina=g}
  else raise(InvalidanDatum("Datum nije validan!"))

let datum_to_string({dan=d;mjesec=m;godina=g}) =
  let ime_mjeseca =
    match m with
    | 1 -> "Januar"
    | 2 -> "Februar"
    | 3 -> "Mart"
    | 4 -> "April"
    | 5 -> "Maj"
    | 6 -> "Juni"
    | 7 -> "Juli"
    | 8 -> "August"
    | 9 -> "Septembar"
    | 10 -> "Oktobar"
    | 11 -> "Novembar"
    | 12 -> "Decembar"
    | _ -> failwith("Nije validan mjesec")
  in
  string_of_int(d) ^ ". " ^ ime_mjeseca ^ " " ^ string_of_int(g) ^ "."

let koji_dan({dan=d;mjesec=m;godina=g}) =
  let pomjeren_m =
    if m >= 3 && m <=12
    then m
    else if m = 1 then 13
    else 14
  in
  let k = g mod 100
  in
  let j = g / 100
  in
  let dan_u_sedmici = (7 + (d + (13*(pomjeren_m+1)))/5 + k + k / 4 + j / 4 -
2*j) mod 7) mod 7
  in
  match dan_u_sedmici with
  | 0 -> "Subota"
  | 1 -> "Nedjelja"
  | 2 -> "Ponedjeljak"
  | 3 -> "Utorak"
  | 4 -> "Srijeda"
  | 5 -> "Cetvrtak"
  | 6 -> "Petak"
  | _ -> string_of_int(dan_u_sedmici)

end;;

Datum.datum_to_string(Datum.kreiraj_datum(10, 3, 2020));; (* = "10. Mart 2020."
*)

```

```
Datum.kreiraj_datum(32, 3, 2020);; (* baca  
InvalidanDatum *)  
Datum.koji_dan(Datum.kreiraj_datum(23, 5, 2001));; (* = "Srijeda" *)
```

Ideja: Signatura `DatumTip` definise javni interfejs — tip `datum` je apstraktan (korisnik ne zna kako je implementiran). Jedini nacin kreiranja datuma je `kreiraj_datum`, koje uvijek prolazi kroz `validan_datum`. Pomocne funkcije `broj_dana_u_mjesecu` i `validan_datum` su u `struct`-u ali **nisu** u `sig`-u — automatski su privatne.

Zadaci za vjezbu

V1. Prosiriti modul `Datum` (signaturu i implementaciju) sa sljedećim funkcijama:

- `koliko_dana_izmedju: datum -> datum -> int` — vraca apsolutni broj dana izmedju dva datuma
- `uvecaj_datum: datum -> int -> datum` — vraca datum uvecan za dati broj dana

V2. Implementirati modul i module type (signaturu) za klasu `RGBColor`. Svaka boja ima tri atributa `r`, `g` i `b` koji su cijeli brojevi izmedju 0 i 255. Unutar modula definisati `type color` za boju. Onemogućiti korisniku pravljenje boja sa nevalidnim atributima. Pored toga modul treba sadržavati:

- `red: color, green: color, blue: color` — predefinisane boje (**public**)
- `max_255: int -> int` — vraca proslijedjeni parametar ako je manji ili jednak 255, u suprotnom vraca 255 (**private**)
- `add: color -> color -> color` — sabira dvije boje; ako zbir nekog atributa prelazi 255, uzima se 255 (**public**)
- `darken: int -> color -> color` — dijeli svaki atribut boje sa proslijedjenim prirodnim brojem (**public**)
- `lighten: int -> color -> color` — mnozi svaki atribut boje sa proslijedjenim prirodnim brojem; ako atribut predje 255, uzima se 255 (**public**)
- `mix_colors: (color * int * (int -> color -> color)) list -> color` — prima listu čiji elementi sadrže boju, prirodan broj i funkciju `darken` ili `lighten`; vraca boju dobijenu kada se prvo primijeni `darken/lighten` sa odgovarajucim parametrima na svaku boju, a zatim se one saberu. Npr. za listu `[(c1, i1, darken); (c2, i2, lighten)]` rezultat je `add (darken i1 c1) (lighten i2 c2)`. Obavezno koristiti `List.fold_left`. (**public**)
- Demonstrirati poziv `mix_colors` van modula (**public**)
- Demonstrirati kako napraviti binding `purple` van modula koji predstavlja boju `r=127, g=0, b=127`